

01. Giới thiệu Spring Boot Framework

1.1. Spring Boot là gì?

Spring Boot là một framework mã nguồn mở được phát triển bởi Pivotal (nay là VMware) nhằm đơn giản hóa quá trình phát triển ứng dụng Spring. Nó được xây dựng dựa trên nền tảng của Spring Framework truyền thống, nhưng loại bỏ phần lớn cấu hình phức tạp và boilerplate code, giúp các nhà phát triển tạo ra các ứng dụng độc lập, sẵn sàng sản xuất (production-ready) một cách nhanh chóng và dễ dàng.

Trong bối cảnh phát triển phần mềm hiện đại, đặc biệt là với sự trỗi dậy của kiến trúc microservices, việc khởi tạo và cấu hình các ứng dụng Spring truyền thống có thể tốn nhiều thời gian và công sức. Spring Boot ra đời để giải quyết vấn đề này bằng cách cung cấp một cách tiếp cận

dựa trên quan điểm (opinionated), đưa ra các cấu hình mặc định thông minh và hợp lý, giúp bạn bắt đầu dự án chỉ trong vài phút.

1.2. Các tính năng chính của Spring Boot

Spring Boot có nhiều tính năng mạnh mẽ giúp nó trở thành một lựa chọn hàng đầu cho việc phát triển ứng dụng Java:

- **Tự động cấu hình (Auto-configuration):** Đây là tính năng cốt lõi của Spring Boot. Nó tự động cấu hình ứng dụng của bạn dựa trên các dependency có trong classpath. Ví dụ, nếu bạn thêm `spring-boot-starter-web` vào dự án, Spring Boot sẽ tự động cấu hình Tomcat và Spring MVC. Nếu bạn thêm `spring-boot-starter-data-jpa` và một driver cơ sở dữ liệu, nó sẽ tự động cấu hình `DataSource` và `EntityManagerFactory`.
- **Spring Boot Starters:** Là các dependency tiện lợi giúp đơn giản hóa việc quản lý các thư viện. Thay vì phải thêm từng dependency một, bạn chỉ cần thêm một Starter duy nhất, và nó sẽ tự động kéo về tất cả các dependency cần thiết cho một tính năng cụ thể (ví dụ: `spring-boot-starter-web`, `spring-boot-starter-security`).

- **Máy chủ nhúng (Embedded Server):** Spring Boot cho phép bạn đóng gói ứng dụng cùng với một máy chủ nhúng (Tomcat, Jetty, hoặc Undertow) vào một file JAR duy nhất. Điều này giúp ứng dụng của bạn trở nên độc lập và dễ dàng triển khai, chỉ cần chạy bằng lệnh `java -jar .`
- **Externalized Configuration:** Cho phép bạn tách biệt cấu hình ứng dụng khỏi mã nguồn, giúp dễ dàng thay đổi hành vi của ứng dụng trong các môi trường khác nhau (dev, test, prod) mà không cần đóng gói lại ứng dụng. Cấu hình có thể được quản lý thông qua các file `application.properties` hoặc `application.yml`, biến môi trường, đối số dòng lệnh, v.v.
- **Spring Boot Actuator:** Cung cấp các điểm cuối (endpoints) sẵn sàng cho môi trường sản xuất, cho phép bạn giám sát và quản lý ứng dụng của mình khi nó đang chạy (ví dụ: kiểm tra sức khỏe, xem metrics, thông tin môi trường).
- **Spring Boot CLI (Command Line Interface):** Một công cụ dòng lệnh cho phép bạn phát triển ứng dụng Spring Boot một cách nhanh chóng bằng cách sử dụng Groovy, giúp giảm thiểu boilerplate code.

1.3. So sánh Spring, Spring MVC và Spring Boot

Để hiểu rõ hơn về Spring Boot, chúng ta cần phân biệt nó với Spring Framework và Spring MVC.

Tiêu chí	Spring Framework	Spring MVC	Spring Boot
Mục đích	Cung cấp một nền tảng toàn diện cho việc phát triển ứng dụng Java, với các tính năng như Dependency Injection (DI), Aspect-Oriented Programming (AOP), Transaction Management.	Là một module của Spring Framework, cung cấp một framework Model-View-Controller (MVC) để xây dựng các ứng dụng web.	Là một phần mở rộng của Spring Framework, giúp đơn giản hóa việc phát triển và triển khai ứng dụng Spring.
Cấu hình	Yêu cầu cấu hình thủ công và chi tiết thông	Yêu cầu cấu hình <code>DispatcherServlet</code> ,	Cung cấp tự động cấu hình,

	qua XML hoặc Java-based configuration.	ViewResolver , và các thành phần web khác.	giảm thiểu nhu cầu cấu hình thủ công.
Máy chủ	Không cung cấp máy chủ nhúng. Cần triển khai ứng dụng dưới dạng WAR trên một máy chủ ứng dụng bên ngoài (Tomcat, JBoss).	Tương tự Spring Framework.	Cung cấp máy chủ nhúng (Tomcat, Jetty, Undertow), cho phép đóng gói ứng dụng thành một file JAR độc lập.
Quản lý Dependency	Yêu cầu quản lý thủ công các dependency và phiên bản của chúng.	Tương tự Spring Framework.	Sử dụng Spring Boot Starters để đơn giản hóa việc quản lý dependency.
Tóm lại	Là nền tảng cốt lõi.	Là một module để xây dựng ứng dụng web trên nền tảng Spring.	Là một công cụ giúp xây dựng ứng dụng Spring một cách nhanh chóng và dễ dàng hơn.

1.4. Tại sao nên sử dụng Spring Boot?

- **Tăng tốc độ phát triển:** Giảm thiểu thời gian thiết lập và cấu hình dự án, giúp bạn tập trung vào logic nghiệp vụ.
- **Đơn giản hóa quản lý dependency:** Với Spring Boot Starters, bạn không còn phải lo lắng về việc tìm kiếm và quản lý các dependency tương thích.
- **Dễ dàng triển khai:** Khả năng đóng gói ứng dụng thành một file JAR độc lập giúp việc triển khai trở nên cực kỳ đơn giản.
- **Hệ sinh thái phong phú:** Tích hợp tốt với hệ sinh thái Spring rộng lớn (Spring Data, Spring Security, Spring Cloud, v.v.), cung cấp các giải pháp cho hầu hết các vấn đề phát

triển ứng dụng.

- **Phù hợp với Microservices:** Các tính năng như máy chủ nhúng, cấu hình bên ngoài và Actuator làm cho Spring Boot trở thành một lựa chọn lý tưởng để xây dựng các microservices.
- **Cộng đồng lớn và hỗ trợ mạnh mẽ:** Spring Boot có một cộng đồng người dùng đông đảo và được hỗ trợ tích cực bởi VMware, đảm bảo rằng nó luôn được cập nhật và cải tiến.

1.5. Cấu trúc một dự án Spring Boot cơ bản

Một dự án Spring Boot điển hình được tạo ra từ Spring Initializr (<https://start.spring.io/>) có cấu trúc như sau:

Plain Text

```
my-project
├── .mvn
│   └── wrapper
│       └── ...
├── src
│   ├── main
│   │   ├── java
│   │   │   ├── com
│   │   │   │   ├── example
│   │   │   │   │   └── myproject
│   │   │   │       └── MyProjectApplication.java // Lớp ứng dụng chính
│   │   └── resources
│   │       ├── static/ // Chứa các file tĩnh (CSS, JS,
│   │       │   images)
│   │       ├── templates/ // Chứa các template (Thymeleaf,
│   │       │   FreeMarker)
│   │       └── application.properties // File cấu hình chính
│   └── test
│       ├── java
│       │   ├── com
│       │   │   ├── example
│       │   │   │   └── myproject
│       │   │       └── MyProjectApplicationTests.java // Lớp kiểm thử
└── mvnw
```

```
├─ mvnw.cmd
└─ pom.xml
```

// File cấu hình Maven

- **MyProjectApplication.java** : Đây là điểm khởi đầu của ứng dụng, chứa `main` method và annotation `@SpringBootApplication` .
- **@SpringBootApplication** : Là một annotation tiện lợi, kết hợp ba annotation khác:
 - `@Configuration` : Đánh dấu lớp này là một nguồn định nghĩa bean cho `ApplicationContext` .
 - `@EnableAutoConfiguration` : Kích hoạt cơ chế tự động cấu hình của Spring Boot.
 - `@ComponentScan` : Quét các component (controllers, services, repositories) trong package hiện tại và các package con.
- **application.properties** : File cấu hình chính của ứng dụng, nơi bạn có thể định nghĩa các thuộc tính như cổng máy chủ, cấu hình cơ sở dữ liệu, v.v.
- **pom.xml** : File cấu hình của Maven, nơi bạn quản lý các dependency (bao gồm cả Spring Boot Starters) và các plugin build.

Bằng cách cung cấp một cấu trúc dự án rõ ràng và các cấu hình mặc định thông minh, Spring Boot giúp bạn nhanh chóng xây dựng và phát triển các ứng dụng Java mạnh mẽ và có khả năng mở rộng.